

07

Retrieval Augmented Generation (RAG)

Building Knowledge-Aware AI Systems

IN THIS WEEK · 3 TOPICS · 35 SECTIONS

01

RAG Foundations

13 SECTIONS

- 01 The Big Picture First
- 02 What is RAG
- 03 Why RAG Exists
- 04 Vector Embeddings
- 05 Embedding Models
- 06 Semantic Search
- 07 Similarity Search
- 08 Chunking Strategies
- 09 Metadata Filtering
- 10 Hybrid Search
- 11 Sparse vs Dense Retrieval
- 12 Context Assembly
- 13 Grounding AI Responses

02

Vector Databases

11 SECTIONS

- 01 Vector Database Architecture
- 02 Pinecone
- 03 Qdrant
- 04 ChromaDB
- 05 Weaviate
- 06 Milvus
- 07 FAISS
- 08 Index Types
- 09 ANN Search
- 10 Collection Design
- 11 Vector Storage Optimization

03

Advanced RAG

11 SECTIONS

- 01 Multi-Document RAG
- 02 Parent-Child Retrieval
- 03 Recursive Retrieval
- 04 Query Transformation
- 05 Re-ranking
- 06 Context Compression
- 07 Citation Generation
- 08 Multi-Modal RAG
- 09 Graph RAG
- 10 Agentic RAG
- 11 RAG Evaluation

WEEK 07 OF 17

01

02

03

04

05

06

07

08

09

10

11

12

13

14

15

16

17

RAG Foundations

The Big Picture First

Before diving into each piece, it helps to understand the core problem RAG is trying to solve. Large language models like GPT or Claude are trained on a huge snapshot of text up to a certain point in time, and then that training essentially "freezes." The model doesn't automatically know about your company's internal documents, it doesn't know about events that happened after its training ended, and it doesn't know about anything private or specific to you. RAG is the technique the industry developed to fix this gap — it lets you connect a language model to your own external knowledge, in real time, without having to retrain the model itself. With that framing in mind, let's go through each concept one at a time.

01 What is RAG

RAG stands for Retrieval-Augmented Generation. That's a mouthful, so let's break the phrase into its three pieces because each word actually tells you exactly what's happening.

"Retrieval" means fetching relevant information from somewhere — think of it like searching through a filing cabinet or a library to pull out the specific pages that are relevant to a question. "Augmented" means enhanced or added to — the information you retrieved gets added into the mix. "Generation" refers to the AI model actually writing a response, the way it normally would.

Put together, RAG is a technique where, before the AI model answers your question, the system first searches through a knowledge base (like a collection of documents, a database, or a set of web pages) to find the most relevant pieces of information, and then hands those pieces to the AI model along with your original question. The model then generates its answer using both its own general knowledge and the specific information it was just handed.

Think of it like an open-book exam versus a closed-book exam. A language model normally answers questions "closed-book" — purely from what it memorized during training. RAG turns it into an "open-book exam" — right before answering, it gets to quickly flip through the exact right pages of a reference book that are relevant to the question, and then write its answer using what it just read, combined with what it already knew.

02 Why RAG Exists

To really appreciate why RAG became such a widely used technique, it helps to understand the specific problems it solves, because there are actually several distinct ones.

Problem one: knowledge cutoffs. Every AI model has a training cutoff date — a point in time after which it simply has no information, because it hasn't seen any text written after that. If you ask a model about something that happened last week, it has no way of knowing, because that event didn't exist yet when the model was trained. RAG solves this by retrieving current information (like a fresh news article or an updated webpage) at the moment you ask, so the model can incorporate up-to-date facts even though its own training is frozen in the past.

Problem two: private or specific knowledge. A company's internal documents, a person's personal notes, a specific product's technical manual — none of this was part of the model's training data, because it's private and was never published on the public internet. RAG lets you feed exactly this kind of private information to the model at the moment of asking, without needing to retrain the entire model (which would be extraordinarily expensive and slow) every time a document changes.

Problem three: hallucination. This is a well-known weakness of language models — when they don't actually know something, instead of saying "I don't know," they sometimes confidently generate an answer that sounds correct but is actually made up. This happens because the model is fundamentally a text-prediction system, not a database of verified facts. RAG reduces this problem significantly, because the model is explicitly given real source material to base its answer on, rather than having to rely purely on its fuzzy, compressed memory of its training data.

Problem four: cost and speed of updating knowledge. Retraining a large language model from scratch (or even partially) every time new information becomes available is incredibly expensive and time-consuming — it can take weeks and cost millions of dollars for the largest models. RAG sidesteps this entirely. Updating the model's effective "knowledge" is as simple as adding a new document to the knowledge base it searches through — no retraining required at all.

Together, these four problems explain why RAG became one of the most widely adopted patterns in real-world AI applications — it's dramatically cheaper and faster than retraining, and it makes answers more accurate and current.

03 Vector Embeddings

This is one of the more technical-sounding concepts, but the core idea is actually pretty intuitive once you see it laid out.

Computers are fundamentally good at working with numbers, not with the *meaning* of words. If you give a computer the word "dog" and the word "puppy," it doesn't inherently know these words are closely related in meaning — to a computer, they're just two completely different sequences of letters. Vector embeddings solve this by converting words, sentences, or entire documents into long lists of numbers (called "vectors") in such a way that the numbers actually capture the *meaning* of the text.

Here's the key trick: this conversion process is designed so that pieces of text with similar meanings end up with similar numbers, and pieces of text with very different meanings end up with very different numbers. So the vector (list of numbers) for "dog" and the vector for "puppy" would end up quite close to each other, while the vector for "dog" and the vector for "airplane" would end up quite far apart.

You can think of this as plotting every word or sentence as a point in space — imagine an enormous multi-dimensional map (way more than the three dimensions we can visualize, often hundreds or even thousands of dimensions) where similar concepts naturally cluster near each other, like how all the coffee shops on a real map might cluster together in a particular part of a city. "King" and "Queen" would land near each other on this conceptual map. "Car" and "vehicle" would land near each other too. This numeric representation of meaning is called an embedding, and it's the foundational building block that makes the rest of RAG possible, because it gives computers a way to mathematically compare how similar two pieces of text are in meaning, not just in exact wording.

04 Embedding Models

If vector embeddings are the numeric representation of meaning, embedding models are the actual AI systems responsible for creating those numbers in the first place. An embedding model is a specialized type of AI model whose entire job is to take a piece of text as input and output a vector (that list of numbers) representing its meaning.

It's worth understanding that embedding models are different from the general chat-style language models like GPT or Claude that you might be more familiar with. A chat model's job is to have a conversation and generate new text. An embedding model's job is much narrower and more specialized — it doesn't generate any new text at all, it simply reads text and converts it into that meaningful numeric vector. Because the job is narrower, embedding models tend to be smaller, faster, and much cheaper to run than full conversational AI models.

There are many embedding models available from different providers — OpenAI has its own embedding models, as does Google, Cohere, and there are also many high-quality open-source options that you can run yourself. Different embedding models vary in a few important ways: the size of the vector they produce (some produce shorter lists of numbers, some produce longer ones — longer generally captures more nuance but costs more to store and search), how well they've been trained to understand the specific type of content you care about (some are better tuned for general text, others are specialized for code, legal documents, or multiple languages), and their cost and speed.

An important practical detail: whichever embedding model you choose to convert your documents into vectors, you must use that exact same model later when converting a user's question into a vector too, because different embedding models produce numbers on different "scales" that aren't directly comparable to each other — it would be like trying to compare a distance measured in miles to a distance measured in kilometers without converting between them first.

05 Semantic Search

Now that we understand embeddings, we can talk about what they're actually used for — and the biggest use case is something called semantic search.

Traditional search, the kind you might be used to from a basic search box, generally works by matching exact keywords. If you search for "car repair" in a traditional keyword search system, it looks for documents that literally contain the words "car" and "repair." If a highly relevant document instead uses the words "automobile maintenance," a simple keyword search might completely miss it, even though it's talking about exactly the same topic — because it doesn't contain those exact keywords.

Semantic search solves this problem directly, using the embeddings we just discussed. Instead of matching exact words, semantic search converts your query into a vector (using an embedding model) and then finds documents whose vectors are closest to it in that numeric "meaning space" we talked about earlier. Since "car repair" and "automobile maintenance" have very similar meanings, their vectors end up close together, so semantic search successfully finds the relevant document even though not a single word matches exactly.

This is a genuinely big leap forward for search quality, because it means the system is actually searching based on *what you mean*, not just the literal words you happened to type. It handles synonyms naturally, it handles different phrasings of the same underlying question, and it even works reasonably well across

different ways of describing the same concept, all without anyone having to manually program in every possible synonym or rephrasing.

06 Similarity Search

Similarity search is very closely related to semantic search — in fact, it's the underlying mechanical process that makes semantic search actually work. Where "semantic search" describes the overall goal (finding results based on meaning), "similarity search" describes the specific mathematical operation being performed behind the scenes.

Here's what's actually happening. Once you have a big collection of vectors (remember, each one represents the meaning of a chunk of text) stored somewhere, and you have a new vector representing your search query, similarity search is the process of mathematically comparing your query vector against every stored vector to figure out which ones are closest, or most "similar," to it.

There are a few common mathematical methods used to measure this "closeness," but the most common one you'll hear about is called cosine similarity. Without getting too deep into the math, the basic idea is that it measures the angle between two vectors — if two vectors are pointing in almost exactly the same direction in that multi-dimensional space we talked about earlier, they're considered highly similar, and if they're pointing in very different directions, they're considered dissimilar. The output is typically a score, often between 0 and 1 (or -1 and 1 depending on the method), where a higher score means more similar meaning.

In practice, when a RAG system searches through thousands or even millions of stored document chunks, it calculates a similarity score for each one against your question, and then returns the ones with the highest scores — usually the "top K" results, meaning something like the top 3, 5, or 10 most similar chunks, since handing the AI model absolutely everything would be slow, expensive, and unnecessary.

07 Chunking Strategies

Here's a practical problem that comes up quickly once you start actually building a RAG system: documents are often really long — think of a 100-page company handbook, or a lengthy technical manual. You generally can't (and shouldn't) convert an entire massive document into a single embedding vector, because a single vector trying to represent an enormous amount of varied content ends up being a vague, blurry average that doesn't represent any single part of it well, and it also isn't practical to hand a model an entire document when only one paragraph is actually relevant to the question asked.

The solution is chunking — breaking documents down into smaller, more manageable pieces before converting each piece into its own embedding vector. This way, when someone asks a question, the system can find the one specific chunk that's actually relevant, rather than the whole giant document.

But how exactly should you break a document into pieces? This is trickier than it sounds, and there are several common strategies, each with tradeoffs.

Fixed-size chunking is the simplest approach — you just split the text every certain number of words or characters, like every 500 words, regardless of where sentences or paragraphs naturally end. It's easy to implement, but it can awkwardly cut a sentence or an idea right in half between two chunks, losing important context.

Sentence-based or paragraph-based chunking splits text along natural boundaries instead — at the end of sentences or paragraphs — so you don't end up slicing an idea in half mid-thought. This tends to preserve meaning better than purely fixed-size chunking.

Overlapping chunks is a technique often combined with the above, where each chunk shares a little bit of text with the chunk before and after it (for example, the last 50 words of one chunk are repeated as the first 50 words of the next). This helps make sure that an idea spanning the boundary between two chunks isn't lost or split awkwardly in a way that neither chunk fully captures it.

Semantic chunking is a more sophisticated approach where, instead of splitting by a fixed rule, the system actually tries to detect where the topic or subject naturally shifts in the text, and splits there — keeping each chunk focused on one coherent idea, even if that means chunks end up being different sizes.

Choosing the right chunk size is itself a balancing act. Chunks that are too small might lose important surrounding context (a sentence pulled completely out of context can be confusing or misleading). Chunks that are too large start running back into the original problem — diluting the specific relevant information among a lot of less relevant surrounding text, and making retrieval less precise.

08 Metadata Filtering

So far we've talked entirely about searching based on meaning. But in a lot of real-world situations, you also want to narrow your search using more concrete, structured facts — and that's what metadata filtering is for.

Metadata simply means "data about the data" — extra structured information attached to each chunk of text, beyond just its content. For example, alongside a chunk of text, you might also store metadata like: which document it came from, the date it was written or last updated, the author, the department it belongs to, its language, or its category or tag.

Metadata filtering means narrowing your search results using this structured information, in combination with the semantic/similarity search we talked about earlier. For example, imagine you're building a RAG system for a company's internal knowledge base, and someone asks a question. You might want to search only within documents from the HR department, only within documents updated in the last year, or only within documents marked as "publicly shareable" (excluding anything confidential from certain users). Metadata filtering lets you apply these kinds of precise, rule-based restrictions before or alongside the fuzzy, meaning-based similarity search.

This combination is powerful because pure semantic search alone doesn't understand strict rules well — it can't reliably guarantee "only show me things from after January 2024," because that's not really a matter of meaning, it's a matter of a specific fact. Metadata filtering handles the parts of a query that are about hard facts and structured constraints, while semantic search handles the parts that are about fuzzy, conceptual relevance — and using both together tends to produce far more precise, useful results than either one alone.

09 Hybrid Search

Hybrid search is exactly what it sounds like — combining two different search approaches together to get the benefits of both, rather than relying on just one. Specifically, it usually refers to combining semantic search (the meaning-based approach we discussed above) with traditional keyword search (the exact-word-matching approach).

Why would you want both? Because each one has situations where it clearly outperforms the other, and combining them covers each other's weaknesses. Semantic search is great at understanding intent, synonyms, and rephrasing — but it can sometimes struggle with very specific, exact terms, like a product ID number, an error code, a person's name, or a rare technical acronym, because these specific terms don't necessarily have a rich, well-defined "meaning" in the numeric embedding space the way a common concept like "car repair" does. Traditional keyword search, on the other hand, is extremely reliable at finding exact matches for specific terms like that, but it completely misses relevant results that don't use the exact same wording.

Hybrid search runs both types of search at the same time, and then combines and re-ranks the results from each, so you get the best of both worlds — the conceptual flexibility of semantic search, plus the precision of exact keyword matching. In practice, many production RAG systems today use hybrid search as their default approach, rather than relying purely on semantic search alone, precisely because real-world questions often contain a mix of general concepts and specific, exact terms that both need to be handled well.

10 Sparse vs Dense Retrieval

This topic is closely related to hybrid search, and it's really just a more technical way of describing the same two underlying approaches we just discussed — but it's worth understanding the specific terminology, since you'll see it used a lot in RAG discussions.

Dense retrieval refers to the semantic, embedding-based search approach we've been describing throughout this whole explanation. It's called "dense" because the vectors involved are dense — meaning nearly every number (every dimension) in that long list of numbers holds some meaningful value, contributing in some small way to representing the overall meaning of the text. Dense retrieval is what powers semantic search, and it's excellent at understanding conceptual similarity and intent.

Sparse retrieval refers to more traditional keyword-based methods (like the classic search technique called TF-IDF, or the more modern and refined version called BM25, which is still very widely used today). It's called "sparse" because in this approach, text is represented as a vector where most of the values are literally zero, and only the specific words that actually appear in the document have a non-zero value. Think of it like a checklist of every possible word in the entire language, where a document gets a mark next to only the words it actually contains, and every other word on that enormous checklist is left blank (zero). This is naturally very good at exact matching, but it has no real understanding of meaning or synonyms — to a sparse method, "car" and "automobile" are just two completely unrelated entries on the checklist.

Understanding this distinction helps clarify exactly what hybrid search is doing under the hood — it's literally combining a dense (meaning-based) retrieval method with a sparse (exact keyword-based) retrieval method, and merging their results together, so that the final results benefit from both the conceptual understanding of dense retrieval and the precision of sparse retrieval.

11 Context Assembly

Once your system has actually found the most relevant chunks of information — whether through semantic search, hybrid search, or metadata filtering, or some combination — there's still an important step left before you can hand things off to the AI model to generate its answer, and that's context assembly.

Context assembly is the process of taking all those separately retrieved chunks and organizing them into a single, well-structured package that gets sent to the language model along with the user's original question. This might sound simple, but doing it well actually matters a lot for getting good quality answers.

A few important considerations come into play here. **Ordering matters** — the sequence in which you present the retrieved chunks to the model can influence how well it uses them; some approaches deliberately put the most relevant chunks either at the very beginning or the very end of the context, since models sometimes pay less attention to information buried in the middle of a long input. **Avoiding redundancy matters** — if several of your retrieved chunks say largely the same thing, including all of them wastes valuable space (remember, longer inputs cost more and there's a limit to how much text you can include at all) without adding much new value, so some systems deliberately filter out near-duplicate chunks. **Fitting within limits matters** — every language model has a maximum amount of text it can process in one go, so context assembly needs to make sure the total combined size of the retrieved chunks, plus the user's question, plus any additional instructions, all fit within that limit. **Clear formatting matters** — well-built systems often explicitly label each piece of retrieved information (for example, marking clearly where one document chunk ends and another begins, and citing which source each one came from) so the model can distinguish between different pieces of retrieved context and, importantly, so it can accurately cite its sources back to the user afterward.

Essentially, context assembly is the "packaging" step — taking a pile of individually useful puzzle pieces and arranging them into a clear, organized, well-labeled bundle, in a way that sets the AI model up to actually use them effectively rather than getting confused by a messy jumble of disconnected text.

12 Grounding AI Responses

Finally, let's talk about grounding, which is really the whole point that everything we've covered so far is building toward.

"Grounding" refers to making sure an AI model's response is actually based on, or tied to, real, verifiable information — rather than the model just generating something that sounds plausible purely from its own internal, sometimes fuzzy memory. It's essentially the antidote to hallucination, which we touched on earlier — the tendency of language models to confidently state things that simply aren't true.

Everything discussed in this whole document — retrieving relevant chunks, filtering by metadata, combining sparse and dense search, and carefully assembling the retrieved context — exists to serve this one ultimate purpose: giving the model real, specific, verified source material to "ground" its answer in, instead of letting it rely purely on guesswork from its training. When a RAG system works well, the AI model's final answer should be traceable directly back to specific pieces of the retrieved content, rather than being an ungrounded, made-up response.

In well-designed RAG systems, grounding is often reinforced through explicit instructions given to the model — for example, telling it directly to only answer based on the provided context, and to clearly say "I don't know" or "this isn't covered in the available information" if the retrieved chunks genuinely don't contain an answer to the question, rather than falling back on making something up to avoid seeming unhelpful. Many systems also implement citations, where the model is asked to reference exactly which source document or chunk it pulled each specific piece of information from — which not only improves user trust (since people can verify the answer themselves by checking the original source), but also makes it much easier to catch and correct cases where the grounding didn't work as intended.

Ultimately, grounding is the reason RAG matters so much in real-world, production AI applications — especially for use cases like legal research, medical information, financial data, or internal company knowledge, where a confidently wrong, ungrounded answer isn't just unhelpful, it can be genuinely harmful. RAG, done well, transforms a language model from a system that's "impressively good at sounding right" into one that's "reliably and verifiably right," anchored in real source material rather than a fuzzy, unverifiable memory.

Vector Databases

01 Vector Database Architecture

Let's start by understanding, at a high level, what's actually happening inside a vector database, because this foundation will make everything else in this guide much easier to follow.

At its core, a vector database needs to do three main jobs. First, it needs to **store** vectors — meaning it needs to hold onto potentially millions of these long lists of numbers, along with the original text they represent and any extra metadata (like the document title, date, or category we discussed in the RAG explanation). Second, it needs to **index** those vectors — this is a crucial step we'll dig into more later, but the short version is that it organizes the vectors in a clever internal structure so that searching through them later is fast, rather than having to compare your query against every single vector one by one, which would be painfully slow at large scale. Third, it needs to **search** — when a new query vector comes in, it needs to rapidly find the vectors already stored that are most similar to it, and return them.

Beyond these three core jobs, a well-built vector database architecture typically includes a few other important pieces. There's usually a **metadata store** sitting alongside the vectors, so that when you filter by things like date or category (as we discussed with metadata filtering), the system can quickly narrow down candidates using these structured fields before or during the similarity search, rather than having to check every single stored vector. There's typically an **API layer** — a set of instructions the database understands, like "insert this vector," "search for the closest matches to this vector," or "delete this vector" — which is how your actual application talks to the database. Many production vector databases also have a **distributed architecture**, meaning that instead of running on a single computer, the data and workload are spread across many machines working together. This matters because when you have hundreds of millions of vectors, no single computer has enough memory or processing power to handle it all efficiently, so the database needs to intelligently split ("shard") the data across multiple machines, and combine ("merge") results from each machine when a search is performed.

Finally, most vector databases also handle **persistence** (making sure your data survives even if the computer it's running on restarts or crashes, usually by regularly saving data to disk rather than keeping everything only in fast-but-temporary memory) and **updates** (handling situations where documents in your knowledge base change over time, meaning old vectors need to be updated or deleted, and new ones added, without having to rebuild the entire search index from scratch every time).

02 Pinecone

Pinecone is one of the most well-known vector databases, and it's what's called a "fully managed" service — meaning Pinecone runs and maintains all the actual servers and infrastructure for you, and you simply connect to it over the internet, similar to how you'd use an LLM API. You never have to worry about installing anything, managing servers, or handling scaling yourself — you just send your vectors to Pinecone and query them, and Pinecone handles everything happening behind the scenes.

This makes Pinecone a very popular choice for teams that want to get a RAG system up and running quickly, without needing to hire specialized infrastructure engineers to manage a database themselves. It's built specifically and exclusively for vector search (unlike some of the other options we'll cover, which started as

general-purpose databases and added vector search later), so it tends to be highly optimized and reliable specifically for this use case. Pinecone also handles scaling automatically — as your data grows from thousands to millions of vectors, Pinecone manages the underlying complexity of spreading that data across more infrastructure, without you needing to manually intervene.

The tradeoff with Pinecone is that, being a fully managed commercial service, you pay for convenience — since you're not running the infrastructure yourself, you don't have as much low-level control over exactly how things are configured, and costs can add up as your data grows, compared to some self-hosted alternatives. It's a strong choice for teams that value speed of development and reliability over maximum control or minimizing infrastructure costs.

03 Qdrant

Qdrant (pronounced "quadrant") is a vector database that, like Pinecone, is built specifically and exclusively for vector search — but with an important difference: Qdrant is open-source, meaning its underlying code is publicly available for anyone to inspect, use, and even run themselves for free. Qdrant offers both a self-hosted option (you download it and run it on your own servers or computer) and a managed cloud option (where Qdrant runs it for you, similar to Pinecone, for those who want the convenience without the setup).

Qdrant is well known in the developer community for being fast and efficient, particularly because it's built in a programming language called Rust, which is known for producing very high-performance, memory-efficient software. It also has a reputation for offering rich, flexible filtering capabilities — meaning it's particularly good at combining metadata filtering (like we discussed earlier) together with vector similarity search in a fast and flexible way.

Because it offers a genuinely solid free, self-hosted option, Qdrant tends to appeal to developers and companies that want more control over their infrastructure, want to avoid ongoing subscription costs as their data scales, or have specific requirements (like keeping all data entirely within their own servers for privacy or compliance reasons) that a fully managed third-party service can't easily satisfy.

04 ChromaDB

ChromaDB (often just called "Chroma") is another open-source vector database, and it's particularly well known and loved for being extremely easy and lightweight to get started with. While Pinecone and Qdrant are often used for larger, more production-scale systems, Chroma has built a strong reputation as the go-to choice for developers who are learning RAG for the first time, prototyping quickly, or building smaller-scale applications.

A big part of Chroma's appeal is its simplicity — you can literally get a vector database running with just a few lines of code, directly inside a Python script, without needing to set up any separate server or sign up for any account at all. It can run entirely in memory or save data locally to a file on your own computer, which makes it perfect for experimentation, learning, small projects, or the early prototyping phase of a larger project before you're ready to invest in more heavyweight infrastructure.

The tradeoff is that Chroma, especially in its simplest form, isn't typically designed to handle the same massive scale or the same level of high-traffic, distributed, production-grade workloads that something like Pinecone or a well-configured Qdrant cluster can handle. Because of this, a common and very sensible

pattern is: build and test your RAG application using Chroma because it's fast and simple to iterate with, and then, once you're ready to actually launch a large-scale production application, migrate to a more heavyweight system if your scale genuinely requires it.

05 Weaviate

Weaviate is another open-source vector database, and one of the things that sets it apart is that it was built with AI and machine learning deeply integrated into its core design from the start, rather than added on as an afterthought. It offers both self-hosted and managed cloud options, similar to Qdrant.

One standout feature of Weaviate is its built-in support for automatically generating embeddings for you. With some other vector databases, you're expected to generate the vector embeddings yourself (using a separate embedding model, like we discussed earlier) and then hand those already-made vectors to the database to store. Weaviate can optionally handle that embedding-generation step internally too, connecting directly to various embedding model providers, which can simplify the overall setup of a RAG pipeline since you have fewer separate pieces to wire together yourself.

Weaviate also has strong support for hybrid search (combining semantic and keyword search, which we covered in the RAG explanation) built directly into the product, and it supports a rich data model where you can define structured "classes" or types of objects with defined properties, similar in spirit to how you might define tables and columns in a traditional database — which can make it a good fit for applications with more complex, structured data alongside the unstructured text.

06 Milvus

Milvus is an open-source vector database that was specifically built from the ground up with a strong focus on massive scale — handling billions of vectors reliably in demanding, high-traffic, production environments. It's part of a broader open-source foundation and has one of the more mature, battle-tested track records among open-source vector databases, particularly among larger companies with very large-scale AI infrastructure needs.

Milvus has a distributed architecture designed to spread enormous datasets across many machines efficiently, and it supports a wide range of the different index types we'll discuss shortly, giving engineering teams fine-grained control over the tradeoffs between search speed, accuracy, and memory usage. Because of this focus on scale and configurability, Milvus tends to have a steeper learning curve and requires more infrastructure expertise to set up and manage well, compared to something like Chroma's beginner-friendly simplicity.

Milvus is generally the kind of choice a larger engineering team reaches for once they have a clear, demanding, large-scale production requirement, and they have the technical expertise on hand to properly configure and maintain a more complex distributed system in exchange for the performance and scale it can deliver.

07 FAISS

FAISS stands for Facebook AI Similarity Search (it was originally developed by Meta, formerly known as Facebook), and it's a bit different in category from everything else we've discussed so far. FAISS is not really a full "database" in the same sense as Pinecone, Qdrant, Weaviate, or Milvus — it doesn't have things like a

network API you connect to, built-in persistence, or metadata filtering out of the box. Instead, FAISS is a software library — a toolkit of highly optimized algorithms for doing fast similarity search on vectors, which developers can use as a building block inside their own applications or even inside other vector databases.

In fact, several of the more full-featured vector databases mentioned above actually use FAISS's algorithms internally as part of their own systems, because FAISS is so well-optimized and widely trusted for the core mathematical problem of searching through huge numbers of vectors quickly.

Because it's a lower-level library rather than a ready-to-use database, FAISS tends to be the choice for developers and researchers who want maximum control and top-tier raw performance, and who are comfortable building the surrounding infrastructure (like storage, metadata handling, and network access) themselves, rather than getting all of that provided out of the box. It's a fantastic option for research projects, highly custom systems, or as the internal engine that powers a bigger, custom-built application — but less commonly the direct choice for someone who just wants to quickly stand up a working RAG system without a lot of extra engineering.

08 Index Types

We've mentioned "indexing" a few times now, so let's dig into what this actually means, because it's one of the most important concepts for understanding how vector databases achieve fast search.

Imagine you have ten million stored vectors, and someone sends in a new query vector. The most straightforward way to find the closest matches would be to compare the query against every single one of those ten million vectors, one at a time, and see which ones are closest. This approach, often called a "flat" or "brute-force" search, is guaranteed to find the truly correct, exact closest matches every time — but it's extremely slow at large scale, because you're doing millions of comparisons for every single search.

An index is a cleverly organized internal data structure that gets built ahead of time, specifically to make searches much faster, by avoiding the need to compare against every single stored vector. There are several common types of indexes used in vector databases, each with different tradeoffs.

Flat index is exactly the brute-force approach described above — simple, and perfectly accurate, but slow once you have a lot of vectors. It's mainly practical for smaller datasets where speed isn't a major concern, or as a baseline for comparing accuracy against faster methods.

IVF (Inverted File Index) works by first grouping similar vectors together into clusters (imagine sorting all your vectors into a set of buckets, where each bucket roughly represents a "neighborhood" of similar meaning). When a new search comes in, instead of checking every single vector, the system first figures out which few buckets are most likely to contain the closest matches, and only searches carefully within those buckets — dramatically cutting down the number of comparisons needed.

HNSW (Hierarchical Navigable Small World) is one of the most popular and widely used index types in modern vector databases. Without getting too deep into the technical details, the basic idea is that it builds a multi-layered network of connections between vectors, a bit like a web of shortcuts, where a search can quickly "hop" through a small number of well-chosen connections to rapidly zero in on the closest neighborhood, rather than wandering through the whole dataset. HNSW is generally known for offering an excellent balance of speed and accuracy, which is why it's become such a common default choice across many vector databases.

PQ (Product Quantization) is a different kind of technique, focused specifically on compressing vectors to use much less memory, by approximating the numbers in each vector using a smaller set of representative values, rather than storing the full precision of every number. This is often combined with other index types (like IVF) when you have such an enormous number of vectors that just storing them all in full precision in memory would be prohibitively expensive.

In practice, most vector databases let you choose which index type to use, and the right choice depends on your specific priorities around speed, accuracy, and memory usage, which brings us to the next topic.

09 ANN Search

ANN stands for Approximate Nearest Neighbor, and understanding this concept is key to understanding why vector search can be so fast even at a massive scale.

Let's unpack the term. "Nearest Neighbor" search means finding the vectors that are truly the closest match, mathematically, to your query — this is what the brute-force flat index we just discussed gives you, with perfect, guaranteed accuracy. But "Approximate" means the system is willing to accept results that are very likely correct, or extremely close to correct, but not mathematically guaranteed to be the absolute perfect, exact closest matches every single time.

Why would anyone deliberately accept slightly less-than-perfect accuracy? Because it turns out that, in exchange for giving up a tiny, often barely noticeable amount of accuracy, you can gain an enormous improvement in speed — often searching through millions or billions of vectors in a matter of milliseconds, rather than the seconds or even minutes a perfectly exact brute-force search might take at that scale. All those index types we just discussed (like HNSW and IVF) are examples of ANN techniques — they're specifically designed to very quickly find results that are almost certainly among the true closest matches, without doing the exhaustive, expensive work of checking absolutely everything.

In the vast majority of real-world applications — like RAG systems answering questions, product recommendation systems, or image search — this small tradeoff in perfect accuracy is completely unnoticeable in practice and very much worth it for the massive gain in speed. If your search returns the 4th and 5th closest matches instead of the mathematically perfect 3rd and 4th closest matches, the practical difference in the quality of the final AI-generated answer is usually negligible, while the speed difference between approximate and exact search at large scale can be the difference between an application that feels instant and one that feels painfully slow.

Most vector databases let you tune exactly how "approximate" you're willing to be, giving you a dial to adjust between faster-but-slightly-less-accurate and slower-but-more-accurate, depending on what your specific application needs.

10 Collection Design

A "collection" (sometimes also called an "index" or a "namespace," depending on the specific database, though this can get confusing since we just used "index" to mean something else above) generally refers to a distinct, organized group of vectors within your vector database — conceptually similar to how a "table" works in a traditional database. Collection design refers to the thoughtful planning of how you organize and structure your data across one or more of these collections.

Getting collection design right matters more than beginners often expect, and there are a few important decisions involved. One key decision is **how many collections to use, and how to divide your data among them**. For example, should all your company's documents from every department live in a single big collection, or should you create separate collections for HR documents, engineering documents, and sales documents? There's no single universal right answer — it depends on your use case. Separate collections can improve search speed and relevance (since you're searching a smaller, more focused set of vectors), and can make access control simpler (giving different users permission to search only certain collections). But splitting things up too aggressively can also add complexity, especially if a single user question sometimes needs to draw from multiple collections at once.

Another important decision involves **choosing what metadata fields to include** alongside each vector in a collection, since (as we discussed in the RAG section) well-chosen metadata enables precise filtering later on. Good collection design means thinking ahead about what filtering capabilities you'll actually need in your application (filtering by date? by author? by department? by document type?) and making sure that information is captured as structured metadata from the start, rather than realizing later that you need it and having to go back and reprocess everything.

A third consideration is **planning for how the collection will grow and change over time** — will new documents be added continuously? Will old documents need to be updated or removed? Good collection design anticipates this from the start, rather than being built only around a fixed, unchanging snapshot of data that becomes awkward to maintain once things inevitably need to change.

11 Vector Storage Optimization

As your collection of vectors grows into the millions, simply storing all that data efficiently — separate from the question of searching it quickly — becomes its own important challenge, and there are several established techniques used to keep storage costs and memory usage manageable.

Quantization is one of the most important techniques here, and it directly connects to the Product Quantization index type we mentioned earlier. The core idea is to reduce the precision of the numbers stored in each vector — instead of storing every number with maximum, full precision (which takes up more memory), you store a slightly less precise, compressed approximation of each number. This can dramatically shrink the amount of memory needed to store your vectors — sometimes by a factor of four, eight, or even more — while causing only a very small, often barely noticeable, reduction in search accuracy.

Choosing the right vector dimensionality is another lever. Remember, the embedding model you choose (from our earlier discussion) determines how long each vector is — how many numbers make up the list. Longer vectors generally capture more nuanced meaning, but they also take up more storage space and are slower to search through. Some embedding models offer smaller vector sizes as an option specifically to help control this tradeoff, and choosing an appropriately-sized embedding model for your actual needs, rather than always defaulting to the largest available option, is an important optimization decision.

Tiered storage is a technique where not all your data needs to sit in the fastest, most expensive type of memory at all times. Frequently accessed or recently added vectors might be kept in fast memory (RAM) for instant access, while older or less frequently accessed vectors get moved to slower, cheaper storage (like a hard drive), and only loaded into fast memory when they're actually needed for a search. This mirrors a very common pattern used broadly across computer systems, applied specifically here to vector data.

Regular cleanup and deduplication also matters more than people often expect — over time, especially in systems where documents get updated frequently, it's easy to accumulate outdated, duplicate, or orphaned vectors (for example, old versions of a document that was since updated, where the old vector never got properly deleted). Periodically cleaning these up keeps both storage costs and search accuracy in good shape, since outdated or duplicate content floating around in your collection can also occasionally get retrieved and confuse the final AI-generated answer, undermining the grounding we talked about in the previous RAG explanation.

Taken together, all of these storage optimization techniques exist to solve the same underlying tension that runs through almost every part of vector database design: balancing speed, accuracy, and cost against each other, and making deliberate, informed tradeoffs among the three based on what actually matters most for your specific application.

Advanced RAG

01 Multi-Document RAG

Basic RAG, in its simplest form, is often explained as if you're searching through one big pile of text and pulling out a few relevant chunks. But in reality, most useful RAG systems need to work across many separate documents — sometimes hundreds, thousands, or more — and the answer to a question might require pulling together pieces from several completely different documents at once.

Multi-document RAG refers to this more realistic, more complex scenario, and the techniques built to handle it well. Imagine you're asking a question like "compare our Q1 and Q3 sales performance" — the answer isn't sitting in a single document; it requires retrieving the Q1 sales report and the Q3 sales report separately, and then combining information from both. A naive RAG system that just grabs the "most similar" handful of chunks overall might end up biased toward one document over the other, or might miss one of the needed documents entirely if its chunks don't score quite as highly in the similarity search.

Handling this well often involves being more deliberate about the retrieval process — for example, explicitly making sure the system retrieves a good spread of results across multiple relevant documents rather than just the top handful of chunks overall, or breaking a complex question down into smaller sub-questions (which connects to query transformation, discussed further below) so that each sub-question can be used to search for and retrieve the specific document it actually needs. It also often involves being thoughtful about which document each retrieved chunk came from, and clearly labeling that in the final context handed to the model, so the model can reason accurately about "the Q1 report says X, while the Q3 report says Y," rather than blending information from different sources together in a confusing, unattributed way.

02 Parent-Child Retrieval

This technique addresses a genuine tension we touched on earlier in the chunking discussion: small chunks are great for precise search (a short, focused chunk produces a more accurate embedding), but small chunks often lack enough surrounding context to be genuinely useful once you actually hand them to the AI model to generate an answer.

Parent-child retrieval solves this by essentially separating the "chunk used for searching" from the "chunk used for the final answer." Here's how it works: you break each document into small chunks (called "child" chunks) specifically for the purpose of generating embeddings and running similarity search, since smaller, more focused chunks tend to produce more precise search matches. But alongside each small child chunk, the system also keeps track of a larger surrounding chunk (the "parent") — perhaps the full paragraph, section, or even the whole page that the small child chunk came from.

When a search happens, the system searches using the small, precise child chunks (getting the benefit of accurate matching), but then, instead of handing just that tiny child chunk to the AI model, it retrieves and hands over the larger parent chunk that contains it — giving the model much richer surrounding context to actually work with when generating its answer. This gives you the best of both worlds: the precision of small-chunk search, combined with the richer context of larger passages, without having to compromise on either one.

03 Recursive Retrieval

Recursive retrieval refers to a search process that doesn't stop after just one round of searching, but instead performs multiple rounds, where each round is informed by what was learned in the previous one — searching, then searching again based on what was found, potentially several times over, digging progressively deeper.

Think about how a human researcher actually works when investigating a complicated question. They rarely find the complete answer from their very first search. More often, they do an initial search, read what comes back, notice it references something else important, then do a follow-up search specifically about that new thing, and this process might repeat several times before they've gathered everything needed to properly answer the original question.

Recursive retrieval tries to mimic this same behavior programmatically. A first search might retrieve a broad, general document or a high-level chunk. The system (or the AI model itself) then examines what was retrieved, and if needed, decides to run a follow-up, more specific search — perhaps searching for more detail on a specific term or reference that showed up in the first result, or drilling down from a high-level document into one of its more detailed sub-sections. This is particularly useful for hierarchical content, like a document that has a table of contents, chapters, and sub-sections — a recursive approach might first retrieve the most relevant chapter, and then perform a second, more focused search specifically within that chapter to find the most relevant sub-section, rather than trying to search the entire document at one single level of granularity all at once.

04 Query Transformation

This technique is based on a simple but important observation: the way a user actually types their question is often not the best possible version of that question to use for searching a vector database. Query transformation refers to a family of techniques where, before running the actual retrieval, the system first modifies, rewrites, or expands the user's original question to make the subsequent search more effective.

There are several common approaches. **Query rewriting** involves taking a vague, poorly worded, or overly casual question and rephrasing it into something clearer and more specific before searching — for example, turning "what about the refund thing" into a more complete, specific question like "what is the company's refund policy," which is likely to produce a much better embedding and better search results.

Query expansion involves generating several different phrasings or related versions of the same underlying question, and searching using all of them, then combining the results. This helps catch relevant documents that might use different terminology than the user's exact original wording, addressing some of the same synonym-related weaknesses we discussed with semantic search.

Query decomposition (breaking a complex question into simpler sub-questions) is especially important for complicated, multi-part questions. If someone asks "how did our marketing spend in Europe compare to Asia last quarter, and which region had better ROI," this is really several distinct questions bundled into one. Decomposing it into separate, simpler questions — the European marketing spend, the Asian marketing spend, and the ROI comparison — and running a separate, focused search for each one, tends to produce far more complete and accurate results than trying to search using the single, complex, bundled-up original question.

Hypothetical Document Embeddings (HyDE) is a clever and slightly different technique, worth knowing about — instead of directly embedding the user's question and searching with it, the system first asks the AI model to generate a hypothetical, made-up example of what a good answer might look like, and then embeds and searches using that hypothetical answer instead of the original question. This works because a hypothetical answer often has more overlap in phrasing and vocabulary with the actual real documents you're searching through than the original, often more casually-worded question does.

05 Re-ranking

Re-ranking is a technique that adds an extra quality-control step after the initial retrieval, but before handing the final results off to the AI model to generate its answer.

Here's the problem it solves: the fast similarity search techniques we discussed earlier (the ANN search, HNSW indexes, and so on) are optimized heavily for speed, since they need to search through potentially millions of vectors in milliseconds. To achieve that speed, they generally make some sacrifices in the precision of exactly how they judge similarity. This means the top results that come back from an initial fast search are usually good and relevant, but not necessarily perfectly, precisely ordered from most to least relevant.

Re-ranking addresses this by taking a two-step approach. First, the fast initial search retrieves a somewhat larger batch of candidate results than you'll actually need — say, the top 50 chunks, instead of just the top 5. Then, a second, separate, and more computationally thorough model (called a re-ranker, or sometimes a "cross-encoder") carefully examines each of those 50 candidates one at a time, directly comparing each one against the original question in a much more detailed way than the fast initial search could afford to do, and re-scores them for relevance far more accurately. Finally, only the truly best few results, according to this more careful re-scoring, actually get passed along to the AI model.

Why not just always use this more careful, more accurate method for the entire search from the start? Because it's simply too slow and computationally expensive to run against millions of documents — it's only practical to run against a much smaller shortlist of candidates. Re-ranking gives you the best of both worlds: the speed of fast initial search to narrow down from millions to dozens of candidates, combined with the higher accuracy of a slower, more careful method applied only to that much smaller, manageable shortlist.

06 Context Compression

Context compression addresses a very practical constraint we touched on earlier: every AI model has a limited context window (a maximum amount of text it can process in one request), and even within that limit, sending unnecessarily long, bloated context tends to increase cost and can sometimes actually reduce answer quality, since important details can get buried among a lot of less relevant surrounding text.

Context compression refers to techniques for shrinking down the retrieved chunks before handing them to the AI model, keeping only the parts that are actually relevant to the specific question being asked, and discarding the rest.

One common approach is **extractive compression**, where, out of a larger retrieved chunk, the system identifies and keeps only the specific sentences or passages that are directly relevant to the question, and trims away the surrounding parts that aren't. For example, if a retrieved chunk is a full paragraph but only one sentence in the middle actually answers the question, extractive compression would keep just that sentence (perhaps with a little surrounding context) rather than the entire paragraph.

Another approach is **using a smaller, faster AI model specifically to summarize** each retrieved chunk down to its most essential points before handing it off to the main, more expensive model that will generate the final answer. This trades a small amount of extra processing time and cost (running this extra summarization step) for a meaningfully smaller, more focused final context, which can reduce the cost of the main, more expensive generation step, and can sometimes improve answer quality by removing distracting, irrelevant material.

The overall goal of context compression is captured well by the phrase "signal-to-noise ratio" — you want to maximize the amount of genuinely useful, relevant information the model sees, relative to the total amount of text it has to process, rather than just handing over everything you retrieved regardless of how directly relevant each part actually is.

07 Citation Generation

We touched on citations briefly in the earlier discussion of grounding, but it's worth exploring this idea more fully here, since it's such an important part of building trustworthy, production-quality RAG systems.

Citation generation refers to the practice of having the AI model explicitly reference exactly which source document (and often which specific chunk or passage within that document) it used to support each specific piece of information in its answer — similar to how an academic paper includes footnotes or citations pointing back to its sources.

This matters for a few important reasons. It builds **user trust** — when a user can see exactly where a claim came from, and click through to verify it themselves against the original source, they're much more likely to trust and rely on the answer, compared to an answer that just states things confidently with no way to check them. It **improves accountability and error-catching** — if an AI model does make a mistake or misinterprets a source, having clear citations makes it much easier for anyone reviewing the answer to spot exactly where things went wrong, compared to an unsourced answer where a subtle error might slip by unnoticed. And it directly supports the grounding goal we discussed earlier — a model that's specifically instructed to cite its sources is naturally nudged toward actually using the retrieved material, rather than drifting off into ungrounded, made-up territory, since it has to be able to point to where each specific claim came from.

Implementing good citation generation typically involves clearly labeling each retrieved chunk with an identifiable source reference during the context assembly step (which we discussed in the earlier RAG explanation), and then explicitly instructing the AI model, as part of its prompt, to include these specific source labels alongside each claim it makes in its final answer, rather than leaving citation as an optional afterthought that the model might or might not naturally choose to include.

08 Multi-Modal RAG

Everything we've discussed so far has implicitly assumed we're working with plain text documents. But a huge amount of real-world, useful information isn't just text — it's images, charts, diagrams, tables, audio, and video. Multi-modal RAG extends the whole RAG approach to work across these different types of content, not just text.

The core challenge here is that the embedding process (converting content into that numeric vector representing its meaning) needs a way to handle non-text content too. This is where multi-modal embedding models come in — specialized models trained specifically to convert images (or other types of content) into

vectors that live in the same kind of "meaning space" we discussed earlier, sometimes even in a way that lets you compare an image's vector directly against a text vector, so a text search query can successfully find a relevant image, or vice versa.

A practical example: imagine a RAG system built over a company's technical manuals, which include a mix of written instructions and diagrams. Multi-modal RAG would let a user ask a question like "show me the wiring diagram for this part," and the system could search across both the text and the embedded images to find the specific diagram that's actually relevant, rather than being limited to only searching the written text and missing the visual content entirely.

Multi-modal RAG is a genuinely more complex and technically demanding area than plain text RAG, partly because good multi-modal embedding models are less mature and more specialized than text embedding models, and partly because assembling a final, coherent context that mixes text and images together for the AI model to reason over adds real additional complexity. But it's an increasingly important area, since so much of the world's genuinely useful information is locked up in non-text formats like diagrams, screenshots, charts, and scanned documents.

09 Graph RAG

Graph RAG represents a meaningfully different approach to organizing and retrieving knowledge, compared to everything else we've discussed so far. Instead of representing your knowledge base purely as a big collection of independent text chunks with embeddings, Graph RAG represents knowledge as a "graph" — a network of specific entities (like people, places, companies, or concepts) connected to each other by specific, labeled relationships (like "works for," "located in," or "is a subsidiary of").

Why would this be useful, given everything vector search already offers? Because standard vector-based RAG is genuinely great at answering questions about a single, self-contained piece of information ("what is our refund policy"), but it can struggle with questions that require understanding how multiple, separate pieces of information relate and connect to each other across a broader body of knowledge — for example, "which of our suppliers are also connected to the vendor that was involved in that recent regulatory issue." Answering a question like this well requires actually understanding and traversing relationships between specific entities, not just finding chunks of text that happen to be similar in meaning to the question.

Graph RAG builds this network of entities and relationships (often extracted automatically from a document collection using an AI model that reads through the documents and identifies the key entities and how they're related), and then, when answering a question, it can search and "traverse" this graph structure — following relevant connections between entities — rather than relying purely on similarity search over isolated chunks. This makes Graph RAG particularly well suited to complex, relationship-heavy domains, like understanding organizational structures, tracing connections in a large investigative research project, or answering questions in a heavily interconnected knowledge domain, where genuinely understanding "how things connect" matters as much as understanding "what a specific piece of text says."

10 Agentic RAG

Agentic RAG refers to combining RAG with the broader idea of an "AI agent" — meaning an AI system that doesn't just passively receive a question and immediately generate a response, but instead can make its own decisions about what steps to take, including deciding when and how to search for information, in a more dynamic, multi-step way.

In basic RAG, the process is a fixed, predetermined pipeline: search once, retrieve chunks, hand them to the model, generate an answer, done. Agentic RAG makes this process much more flexible and dynamic. Instead of following one fixed sequence every single time, the AI model itself gets to decide, at each step, what to do next — it might decide the initial search results weren't good enough and choose to search again with a different, refined query, it might decide the question actually requires checking two completely different knowledge bases and search both, it might decide it doesn't even need to search at all because it already knows the answer confidently, or it might decide to use an entirely different tool altogether, like doing a calculation or checking a live database, alongside or instead of a document search.

This connects closely to several of the earlier advanced techniques — an agentic system might use query transformation to decide how to rephrase its search, recursive retrieval to decide whether a follow-up search is needed, and re-ranking to evaluate whether its current results are actually good enough before deciding to proceed to generating a final answer. The key distinguishing idea of "agentic" RAG is that these decisions aren't fixed in advance by the system's programmer — they're made dynamically, in real time, by the AI model itself, based on the specific situation it encounters, giving the whole system much more flexibility to handle complex, unpredictable, multi-step questions well, at the cost of being somewhat less predictable and typically slower and more expensive than a simple, fixed, single-pass RAG pipeline.

11 RAG Evaluation

With all these different techniques and moving pieces, an important practical question naturally arises: how do you actually know if your RAG system is working well? RAG evaluation refers to the methods and metrics used to systematically measure the quality of a RAG system, rather than just informally trying a few questions and eyeballing whether the answers seem reasonable.

Good RAG evaluation generally looks at quality from a few distinct angles, because a RAG system has several different components that can each fail in their own particular way. **Retrieval quality** asks: did the system actually find the right, relevant chunks of information in the first place? If the retrieval step fails to find the genuinely relevant document, it doesn't matter how good the AI model is at generating text — it simply won't have the right information to work with. This is often measured using metrics that check whether the correct, known-relevant documents show up in the retrieved results, and how highly they're ranked.

Faithfulness (also sometimes called groundedness) asks: does the final generated answer actually, accurately reflect what the retrieved source material said, without adding in unsupported claims or distorting the source content? This directly measures how well the system is avoiding hallucination — an unfaithful answer might sound fluent and confident, but it may not actually align with the source material it was supposed to be grounded in.

Answer relevance asks a related but distinct question: does the generated answer actually address what the user asked, even if it's perfectly faithful to the retrieved sources? It's possible for an answer to be completely accurate relative to its sources, and yet still fail to actually address the specific question the user asked, perhaps because the retrieval step brought back subtly wrong information, or because the model wandered off-topic while generating its response.

In practice, teams evaluate these dimensions in a few common ways. **Human evaluation** involves people manually reviewing a sample of the system's answers and rating them, which tends to be the most reliable and trustworthy method, but is slow and expensive to do at scale. **Using another, separate AI model as a judge** has become an increasingly popular approach, where a capable language model is given the question,

the retrieved context, and the generated answer, and asked to score things like faithfulness and relevance according to a clear set of criteria — this scales much better than pure human evaluation, though it's generally considered somewhat less reliable than careful human judgment. And **automated benchmark datasets**, made up of predefined questions with known correct answers and known relevant source documents, allow for consistent, repeatable testing as you make changes to your system over time, helping you clearly see whether a specific change (like switching to a different embedding model, or adding re-ranking) actually improved things or made them worse, rather than relying on a vague, subjective sense of "it feels better now."

Ultimately, RAG evaluation matters because a RAG system has many moving, interacting parts — chunking strategy, embedding model choice, retrieval method, re-ranking, prompt design — and without systematic evaluation, it's genuinely difficult to know which specific piece is working well, which piece is causing problems, and whether a change you're considering will actually make the overall system better or worse.